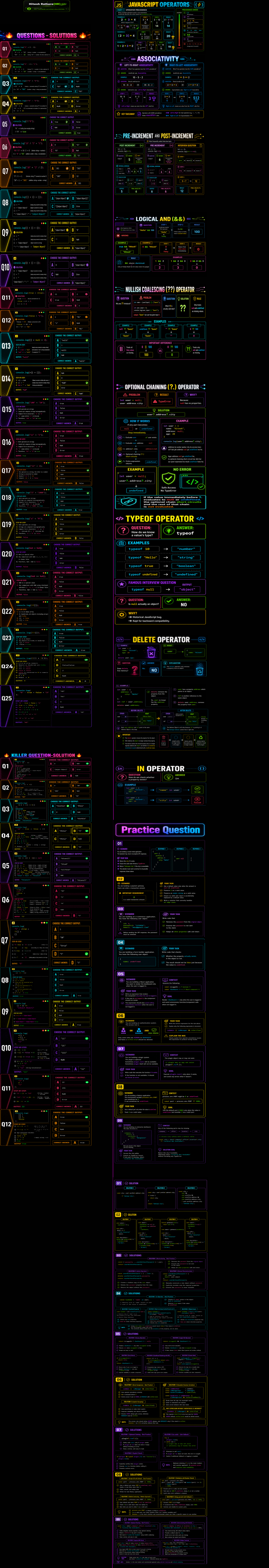

Hitesh Rathore(HR) Notes

🔥 QUESTIONS - SOLUTIONS 🔥

Q1
```
console.log("5" + 2 - 1);
```
SOLUTION:
"5" + 2 → "52" (string + number → concatenation)
"52" - 1 → 51 (string "52" → number 52, then subtract 1)

CHOOSE THE CORRECT OUTPUT:
A 51 ✓ | B "51" | C 52 | D NaN
CORRECT ANSWER: A 51

Q2
```
console.log("5" - 2 + "1");
```
SOLUTION:
"5" - 2 → 3 (string "5" → number 5, then subtract 2)
3 + "1" → "31" (number + string → concatenation)

CHOOSE THE CORRECT OUTPUT:
A 31 ✓ | B "31" | C 4 | D NaN
CORRECT ANSWER: A 31

Q3
```
console.log(+"5" + "2");
```
SOLUTION:
+"5" → 5 (unary + converts string "5" to number 5)
5 + "2" → "52" (number + string → string concatenation)

CHOOSE THE CORRECT OUTPUT:
A 7 | B "7" | C "52" ✓ | D NaN
CORRECT ANSWER: C "52"

Q4
```
console.log(+"5" - "2");
```
SOLUTION:
+"5" → 5 (unary + converts string "5" to number 5)
"2" → 2 ("2" is converted to number 2 in subtraction)
5 - 2 → 3

CHOOSE THE CORRECT OUTPUT:
A 3 ✓ | B "3" | C NaN | D Error
CORRECT ANSWER: A 3

Q5
```
console.log(!!"0");
```
SOLUTION:
"0" → truthy (non-empty string)
!!"0" → true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q6
```
console.log("10" / "2" + "3");
```
SOLUTION:
"10" / "2" → 10 / 2 → 5 (division: strings → numbers)
5 + "3" → "53" (addition: number + string → concatenation)

CHOOSE THE CORRECT OUTPUT:
A 8 | B "53" ✓ | C "83" | D NaN
CORRECT ANSWER: B "53"

Q7
```
console.log("10" + "2" / 2);
```
SOLUTION:
"2" / 2 → 1 (division: string "2" converts to number 2)
"10" + 1 → "101" (addition: string + number → string)

CHOOSE THE CORRECT OUTPUT:
A 101 ✓ | B "101" | C "102" | D 12
CORRECT ANSWER: A 101

Q8
```
console.log([] + {} + []);
```
SOLUTION:
[] → "" (empty array converts to empty string)
{} → "[object Object]" (object converts to string)
[] → "" (empty array converts to empty string)
"" + "[object Object]" + "" → "[object Object]"

CHOOSE THE CORRECT OUTPUT:
A "[object Object]" ✓ | B "[object Object][]" | C "[][object Object]" | D Error
CORRECT ANSWER: A "[object Object]"

Q9
```
console.log([] + [] + {});
```
SOLUTION:
[] → "" (empty array converts to empty string)
[] → "" (empty array converts to empty string)
{} → "[object Object]" (object converts to string)
"" + "" + "[object Object]" → "[object Object]"

CHOOSE THE CORRECT OUTPUT:
A "[object Object]" ✓ | B "0" | C "" | D NaN
CORRECT ANSWER: A "[object Object]"

Q10
```
console.log({} + [] + []);
```
SOLUTION:
{} → "[object Object]" (object converts to string)
[] → "" (empty array converts to empty string)
[] → "" (empty array converts to empty string)
"[object Object]" + "" + "" → "[object Object]"

CHOOSE THE CORRECT OUTPUT:
A 0 | B "[object Object]" ✓ | C NaN | D Error
CORRECT ANSWER: B "[object Object]"

Q11
```
console.log(+true + +true);
```
SOLUTION:
true → 1 (true converts to 1)
+true → +1 → 1
1 + 1 → 2

CHOOSE THE CORRECT OUTPUT:
A 11 | B 2 ✓ | C true | D NaN
CORRECT ANSWER: B 2

Q12
```
console.log(+false + "1");
```
SOLUTION:
false → 0 (false converts to 0)
+false → +0 → 0
0 + "1" → "01"

CHOOSE THE CORRECT OUTPUT:
A 1 | B "01" ✓ | C "1" | D NaN
CORRECT ANSWER: B "01"

Q13
```
console.log([] + null + 1);
```
STEP-BY-STEP:
1) [] → "" (empty array converts to empty string)
2) "" + null → "null" (null converts to "null" in string concatenation)
3) "null" + 1 → "null1" (1 converts to "1")

OUTPUT: "null1"

CHOOSE THE CORRECT OUTPUT:
A "null1" ✓ | B 1 | C null1 | D NaN
CORRECT ANSWER: A "null1"

Q14
```
console.log(+{} + []);
```
STEP-BY-STEP:
1) +{} → NaN (Object converts to NaN with unary +)
2) [] → "" (empty array converts to empty string)
3) NaN + "" → "NaN" (string concatenation)

OUTPUT: "NaN"

CHOOSE THE CORRECT OUTPUT:
A NaN | B 0 | C "NaN" ✓ | D Error
CORRECT ANSWER: C "NaN"

Q15
```
console.log("100" < "20");
```
STEP-BY-STEP:
1) Both operands are strings
2) JavaScript compares strings lexicographically
3) Compare first characters: '1' vs '2'
4) '1' comes before '2' in Unicode
5) Therefore, "100" < "20" is true

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q16
```
console.log("11" < "2");
```
STEP-BY-STEP:
1) Both operands are strings
2) JavaScript compares strings lexicographically
3) Compare first characters: '1' vs '2'
4) '1' comes before '2' in Unicode
5) Therefore, "11" < "2" is true

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q17
```
console.log("11" < 2);
```
STEP-BY-STEP:
1) One operand is a string, the other is a number
2) String is converted to number
3) Number("11") → 11
4) Compare: 11 < 2 → false

OUTPUT: false

CHOOSE THE CORRECT OUTPUT:
A true | B false ✓ | C NaN | D Error
CORRECT ANSWER: B false

Q18
```
console.log("2" > "1000");
```
STEP-BY-STEP:
1) Both operands are strings
2) Strings are compared lexicographically
3) Compare first characters: '2' vs '1'
4) '2' comes after '1' in Unicode
5) Therefore, "2" > "1000" is true

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q19
```
console.log("a" < "B");
```
STEP-BY-STEP:
1) Both operands are strings
2) Strings are compared lexicographically
3) Compare first characters: 'a' vs 'B'
4) 'a' (Unicode 97) > 'B' (Unicode 66)
5) Therefore, "a" < "B" is false

OUTPUT: false

CHOOSE THE CORRECT OUTPUT:
A true | B false ✓ | C NaN | D Error
CORRECT ANSWER: B false

Q20
```
console.log(NaN < NaN);
```
STEP-BY-STEP:
1) NaN is a special value
2) Any comparison with NaN (<, >, <=, >=) is always false
3) Therefore, NaN < NaN is false

OUTPUT: false

CHOOSE THE CORRECT OUTPUT:
A true | B false ✓ | C NaN | D Error
CORRECT ANSWER: B false

Q21
```
console.log(NaN >= NaN);
```
STEP-BY-STEP:
1) NaN is a special value
2) Any comparison with NaN using relational operators (>, >=, <, <=) is always false
3) Therefore, NaN >= NaN is false

OUTPUT: false

CHOOSE THE CORRECT OUTPUT:
A true | B false ✓ | C NaN | D Error
CORRECT ANSWER: B false

Q22
```
console.log(!![]);
```
STEP-BY-STEP:
1) [] is an empty array
2) In JavaScript, all objects (including arrays) are truthy
3) ![] → false
4) !![] → true

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q23
```
console.log(!!{});
```
STEP-BY-STEP:
1) {} is an object
2) All objects are truthy in JavaScript
3) !{} → false
4) !false → true

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q24
```
console.log(![] + ![]);
```
STEP-BY-STEP:
1) [] is an array (object)
2) Objects in JavaScript are truthy
3) ![] → false
4) ![] → false
5) false + false
6) false is converted to 0 in numeric operation
7) 0 + 0
8) Output → 0

OUTPUT: 0

CHOOSE THE CORRECT OUTPUT:
A 0 ✓ | B falsefalse | C 2 | D NaN
CORRECT ANSWER: A 0

Q25
```
console.log(
  "5" - true + false + "2"
);
```
STEP-BY-STEP:
1) "5" - true + false + "2"
2) true → 1, false → 0
3) "5" - 1 + 0 + "2"
4) "5" - 1 → 5 - 1 → 4
5) 4 + 0 → 4
6) 4 + "2" → "4" + "2"
7) "4" + "2" → "42"

OUTPUT: "42"

CHOOSE THE CORRECT OUTPUT:
A "42" ✓ | B "51" | C 4 | D NaN
CORRECT ANSWER: A "42"

🔥 KILLER QUESTION-SOLUTION 🔥

Q1
```
console.log(
  [] + {} - []
);
```
STEP-BY-STEP: [illegible]

OUTPUT: NaN

CHOOSE THE CORRECT OUTPUT:
A 0 | B NaN ✓ | C "[object Object]" | D Error
CORRECT ANSWER: B NaN

Q2
```
console.log(
  +[] + [] + +[]
);
```
STEP-BY-STEP: [illegible]

OUTPUT: "00"

CHOOSE THE CORRECT OUTPUT:
A 0 | B "00" ✓ | C "000" | D NaN
CORRECT ANSWER: B "00"

Q3
```
console.log(
  ![] + [] + ![]
);
```
STEP-BY-STEP: [illegible]

OUTPUT: "falsefalse"

CHOOSE THE CORRECT OUTPUT:
A "falsefalse" ✓ | B "0false" | C 0 | D NaN
CORRECT ANSWER: A "falsefalse"

Q4
```
console.log(
  "10" - true + false + []
);
```
STEP-BY-STEP:
1) "10" → 10 (string → number)
2) true → 1, false → 0 (boolean → number)
3) [] → "" (empty array → empty string)
4) 10 - 1 → 9
5) 9 + 0 → 9
6) 9 + "" → "9" (number + string → string)
7) Output → "9"

OUTPUT: "9"

CHOOSE THE CORRECT OUTPUT:
A "9false" | B "9" ✓ | C "9false" | D NaN
CORRECT ANSWER: B "9"

Q5
```
console.log(
  [] + [] + ![] + null
);
```
STEP-BY-STEP:
1) [] → "" (empty array → empty string)
2) [] → "" (empty array → empty string)
3) "" + "" → "" (string concatenation)
4) ![] → false ([] is truthy → false)
5) "" + false → "false" (string concatenation)
6) null → "null" (null → string "null" in +)
7) "false" + "null" → "falsenull"

OUTPUT: "falsenull"

CHOOSE THE CORRECT OUTPUT:
A "falsenull" ✓ | B "false0" | C "nullfalse" | D NaN
CORRECT ANSWER: A "falsenull"

Q6
```
console.log(
  +!![] +
  +"5" -
  !false +
  []
);
```
STEP-BY-STEP:
1) !![] → true ([] is truthy → !![] = true)
2) +true → 1 (unary + converts true → 1)
3) +"5" → 5 (unary + converts "5" → 5)
4) 1 + 5 → 6
5) !false → true (false is falsy → !false = true)
6) 6 - true → 5 (true converts to 1)
7) [] → "" (empty array → empty string)
8) 5 + "" → "5" (empty string converts to 0)

OUTPUT: "5"

CHOOSE THE CORRECT OUTPUT:
A "50" | B "5" ✓ | C "4" | D NaN
CORRECT ANSWER: B "5"

Q7
```
console.log(
  "5" - +true +
  !![] +
  []
);
```
STEP-BY-STEP:
1) "5" - +true → "5" - 1 → 4 (+true → 1)
2) !![] → true ([] is truthy, ![] → false, !![] → true)
3) 4 + true → 4 + 1 → 5 (true → 1)
4) [] → "" (empty array → empty string)
5) 5 + "" → "5" (adding empty string converts to string)

OUTPUT: "5"

CHOOSE THE CORRECT OUTPUT:
A 5 | B "50" | C "5true" | D "5" ✓
CORRECT ANSWER: D "5"

Q8
```
console.log(
  [] == ![]
);
```
STEP-BY-STEP:
1) ![] → false
2) [] == false
3) false will be converted to 0, now [] == 0
4) [] will be converted to "", now "" == 0
5) "" will be converted to 0, now 0 == 0
6) 0 == 0 → true

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q9
```
console.log(
  [] + {} ==
  "[object Object]"
);
```
STEP-BY-STEP:
1) [] + {} → "" + "[object Object]"
2) "" + "[object Object]" → "[object Object]"
3) "[object Object]" == "[object Object]" → true

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

Q10
```
console.log(
  +[] +
  +![] +
  +!![] +
  "1"
);
```
STEP-BY-STEP:
① +[] → 0 ([] → "" → 0)
② +![] → +false → 0 ([] is truthy, so ![] → false)
③ +!![] → +true → 1 ([] is truthy, so !![] → true)
④ 0 + 0 + 1 + "1" → 1 + "1"
⑤ 1 + "1" → "11" (String Concatenation)

OUTPUT: "11"

CHOOSE THE CORRECT OUTPUT:
A "11" ✓ | B "21" | C "111" | D NaN
CORRECT ANSWER: A "11"

Q11
```
console.log(
  "2" +
  3 -
  "1"
);
```
STEP-BY-STEP:
① "2" + 3 → "23" (String + Number → String)
② "23" - "1" → 23 - 1 (String - String → Number - Number)
③ 23 - 1 → 22

OUTPUT: 22

CHOOSE THE CORRECT OUTPUT:
A 22 ✓ | B 231 | C NaN | D Error
CORRECT ANSWER: A 22

Q12
```
console.log(
  [] +
  [] ==
  [] ==
  false
);
```
STEP-BY-STEP: [illegible]

OUTPUT: true

CHOOSE THE CORRECT OUTPUT:
A true ✓ | B false | C NaN | D Error
CORRECT ANSWER: A true

JS JAVASCRIPT OPERATORS { ⚡ }

PART 1 OPERATOR PRECEDENCE

When multiple operators exist in one expression, JavaScript decides the order based on precedence rules.

EXAMPLE 1: 2 + 3 * 4 — Which happens first?
(2 + 3) * 4 = 20 VS 2 + (3 * 4) = 14 ✓
STEP 1: 3 * 4 → 12 (Multiplication has higher precedence) → STEP 2: 2 + 12 → 14 → RESULT: 2 + 3 * 4 → 14

EXAMPLE 2: (2 + 3) * 4
STEP 1: 2 + 3 → 5 (Parentheses have highest precedence) → STEP 2: 5 * 4 → 20 → RESULT: 20

EXAMPLE 3: 1 + 2 * 3 ** 2
STEP 1: 3 ** 2 → 9 (Exponentiation first) → STEP 2: 2 * 9 → 18 → STEP 3: 1 + 18 → 19 → RESULT: 19

PRECEDENCE ORDER (HIGHEST → LOWEST)
1. ( ) Parenthesis
2. ** Exponentiation
3. * / % Multiplication, Division, Modulus
4. + - Addition, Subtraction
5. < > <= >= Comparison Operators
6. && Logical AND
7. || Logical OR
8. = += -= *= /= etc. Assignment (Lowest)

ASSOCIATIVITY

1 LEFT-TO-RIGHT ASSOCIATIVITY
QUESTION: What if two operators have the SAME precedence?
ANSWER: JavaScript uses Associativity.
EXAMPLE: 10 - 5 - 2
QUESTION: Should JavaScript do: 10 - (5 - 2) OR (10 - 5) - 2
ANSWER: Subtraction uses Left-to-Right Associativity
STEP 1: 10 - 5 → 5 → STEP 2: 5 - 2 → 3 → RESULT: 3
Left-to-Right means we solve from the LEFT side first.

2 RIGHT-TO-LEFT ASSOCIATIVITY
QUESTION: What if two operators have the SAME precedence?
ANSWER: JavaScript uses Associativity.
EXAMPLE: 2 ** 3 ** 2
QUESTION: Should JavaScript do: (2 ** 3) ** 2 OR 2 ** (3 ** 2)
ANSWER: Exponentiation uses Right-to-Left Associativity
STEP 1: 3 ** 2 → 9 → STEP 2: 2 ** 9 → 512 → RESULT: 512
Right-to-Left means we solve from the RIGHT side first.

KEY TAKEAWAY: Operators with SAME precedence follow ASSOCIATIVITY rules.
→ Left-to-Right for most operators (e.g., +, -, *, /, %)
← Right-to-Left for Exponentiation (**)

PRE-INCREMENT AND POST-INCREMENT

☆ Most frequently asked interview topic.

POST INCREMENT (x++)
```
let x = 5;
console.log(x++);
```
QUESTION: Output? | ANSWER: 5 | WHY? Post Increment means: Use value first, Increment later
INTERNAL WORKING: ① temp = x ② x = x + 1 ③ return temp — temp 5 → x 6 → return 5
STEP BY STEP: ① temp = 5 → ② x = 6 → ③ return 5
OUTPUT: 5 | FINAL VALUE OF x: 6

PRE INCREMENT (++x)
```
let x = 5;
console.log(++x);
```
QUESTION: Output? | ANSWER: 6 | WHY? Pre Increment means: Increase first, Use later
INTERNAL WORKING: ① x = x + 1 ② return x — x 6 → return 6
STEP BY STEP: ① x = 6 → ② return 6
OUTPUT: 6 | FINAL VALUE OF x: 6

INTERVIEW QUESTION
```
let x = 5;
console.log(x++ + ++x);
```
STEP 1: x++ → returns 5 → x becomes 6
STEP 2: ++x → x becomes 7 → returns 7
STEP 3: 5 + 7 → 12
RESULT: 12 | FINAL VALUE OF x: 7

LOGICAL AND (&&)

Most beginners think: && always returns true or false
✗ WRONG

QUESTION: "Hello" && 100
STEP 1: JavaScript checks first operand. "Hello" → Truthy
STEP 2: For &&: If first value is truthy, return second value.
RESULT: 100

EXAMPLE: true && "Hitesh" → "Hitesh"

EXAMPLE: 0 && "Hitesh" — STEP 1: 0 → Falsy — STEP 2: Return first falsy value immediately. — RESULT: 0

RULE: && returns: shortcircuit
(Jha pr finally khade hai vhi value return ho jayegi)

EXAMPLES:
1 && 2 → 2
1 && 2 && 3 → 3
0 && 2 && 3 → 0

NULLISH COALESCING (??) OPERATOR

QUESTION: Why was ?? introduced?

PROBLEM:
```
let name = (userInput) || ("Guest");
```
```
let user_input = 0;
console.log(user_input || "Guest");
```
output: "Guest" but user has given input 0.

QUESTION: What if 0 is actually valid data?
SOLUTION: ??
RULE: Only treats: null and undefined as missing values.

EXAMPLES
null ?? "Guest" → "Guest"
undefined ?? "Guest" → "Guest"
"" ?? "Guest" → ""
0 ?? 100 → 0

IMPORTANT DIFFERENCE
|| Treats all falsy values as missing. 0 || 100 → 100
VS
0 ?? 100 → 0 — ?? Treats only null and undefined as missing.

OPTIONAL CHAINING (?.) OPERATOR

PROBLEM:
```
let user = null;
user.address.city
```
RESULT: TypeError
WHY? Because: null has no properties.

SOLUTION:
```
user?.address?.city
```

HOW IT WORKS
If any part becomes: null or undefined — Stop immediately.
1. Evaluate user ✓ user exists
2. Evaluate user.address — null
3. address?.city — address is null
4. Optional chaining (?.) short-circuits
5. Entire expression returns — undefined

EXAMPLE
```
let user = {
  name: "Hitesh",
  address: null,
  age: 20
}
console.log(user?.address?.city);
```
address ke andar jaakar city ko access karo sirf tab jab address null ya undefined na ho.
Agar address null ya undefined ho, to optional chaining short-circuit kar deti hai aur poori expression ka result undefined hota hai.

EXAMPLE
```
let user = null;
user?.address?.city
```
↓ undefined

NO ERROR — Safe Access, No TypeError

If the value immediately before ?. evaluates to null or undefined, the optional chain short-circuits and the rest of that chain is not evaluated.

</> TYPEOF OPERATOR </>

QUESTION: How do we know a value's type?
ANSWER: typeof

EXAMPLES
typeof 10 → "number"
typeof "Hello" → "string"
typeof true → "boolean"
typeof undefined → "undefined"

FAMOUS INTERVIEW QUESTION
typeof null → OUTPUT "object"

QUESTION: Is null actually an object?
ANSWER: NO

WHY?
→ Historical JavaScript bug.
→ Kept for backward compatibility.

DELETE OPERATOR

EXAMPLE
```
let user = {
  name: "Hitesh",
  age: 19
};
delete user.age;
```
RESULT: user → { name: "Hitesh" }

QUESTION: Does delete free memory?
ANSWER: No. NO
EXPLANATION: The delete operator only removes a property from an object.

EXAMPLE
```
let user = {
  address: {
    city: "Delhi"
  }
};
let ref = user.address;
delete user.address;
```
✓ delete removes the property from the object. So, user no longer has the address property.

1. user has a property address which points to an object.
2. ref is another reference to the same object.
3. delete user.address; removes the property from user.

BEFORE DELETE: STACK (user, ref) → HEAP Address Object { city: "Delhi" }
Both user.address and ref point to the same Address Object in the heap.

AFTER DELETE: STACK (user — address property removed from user, ref) → HEAP Address Object { city: "Delhi" } — Still alive because ref is pointing to it.
The Address Object is still alive because ref is pointing to it. The Garbage Collector cannot free it right now.

IMPORTANT
✓ The delete operator removes the property from the object.
✓ After deletion, the object no longer contains that property.
✓ The memory used by the removed property entry is no longer logically needed by the object, but whether it is immediately reclaimed or reused is determined by the JavaScript engine.

delete property removed → property entry no longer exists on the object → memory used by that property entry no longer needed → memory may be reclaimed or reused by the engine

IN OPERATOR

QUESTION: How do we check whether a property exists?
ANSWER: in

EXAMPLE
```
let user = {
  name: "Hitesh",
  age: 19
};
```
→ "name" in user → true
```
let user = {
  name: "Hitesh",
  age: 19
};
```
→ "city" in user → false

Practice Question

01
SCENARIO: You are building a social media application. The backend may return incomplete API responses.

YOUR TASK
- Return the city if it exists.
- Ensure the code never throws a TypeError.
- Return "Unknown City" if the city is unavailable.
- The solution must work correctly for all possible responses shown above.

RESPONSE 1
```
{
  user: {
    profile: {
      address: {
        city: "Delhi"
      }
    }
  }
}
```
RESPONSE 2
```
{
  user: null
}
```
RESPONSE 3
```
{
}
```

02
SCENARIO: You are building a payment gateway. Users can enter a transaction amount.

IMPORTANT REQUIREMENT: 0 is a valid transaction amount.

YOUR TASK
- Use a default value only when the amount is null or undefined.
- Preserve 0 as a valid value.
- Preserve an empty string as a valid value.
- Ensure no valid user input is accidentally replaced by a default value.
- Write a solution that correctly handles all edge cases.

03
SCENARIO: You are building an e-commerce application. You have the following user object:
```
{
  id: 1,
  password: "123456",
  email: "abc@gmail.com"
}
```
⚠ Before sending the API response, the password must be removed.

YOUR TASK — Write code that:
- Removes the password from the original object.
- Ensures the password is not sent to the client.
- Keeps all other properties safe and intact.

04
SCENARIO: You are building a form builder application. You have the following user object:
```
{
  name: undefined
}
```
YOUR TASK — Write code that checks:
- Whether the property actually exists in the object or not.
- The result should not be false just because the value is undefined.

05
SCENARIO: You are building a React application. You want to render the dashboard only when the user is logged in.

YOUR TASK
- Write an expression that renders the component only when the user is logged in.
- If the user is not logged in, the component should not render.
- The expression should also avoid calling unnecessary functions when the user is not logged in.

CONTEXT: Assume the following:
```
const isLoggedIn = /* boolean */
const <Dashboard /> = /* React component */
```
GOAL: Render <Dashboard /> only when the user is logged in and avoid unnecessary function/component calls when not logged in.

06
SCENARIO: You are building an authentication system. The access rule is:
ADMIN OR MANAGER AND EMAIL VERIFIED
Only users who are ADMIN or MANAGER and have a verified email should be allowed.

YOUR TASK
- Write the correct expression for the rule above.
- Explain why the following expression is incorrect:
```
isAdmin || isManager && isVerified
```
EXPLAIN THE BUG: Clearly explain how operator precedence causes this expression to produce wrong results.

07
SCENARIO: You are building a plugin system. The plugin is optional. Sometimes plugin.run() will be available. Sometimes plugin itself will not be available.

YOUR TASK
- Write code that executes the function if it exists.
- If the function is not available, it should not throw an error.

CONTEXT: The plugin object may or may not exist.
```
// plugin may or may not exist
plugin?.run?.();
```
GOAL: Execute plugin.run() only when it exists, and avoid any errors when it doesn't.

08
SCENARIO: You are building a Node.js application. The environment variable process.env.PORT can sometimes be 0. It can sometimes be undefined.

YOUR TASK
- Use a default port only when the value is undefined.
- Treat 0 as a valid value.

CONTEXT: process.env.PORT might be 0 or undefined.
```
const port = process.env.PORT ?? 5000;
```
GOAL: Use the default port (5000) only when the value is undefined, but consider 0 as a valid port.

09
SCENARIO: You are building an enterprise dashboard. The data structure is:
```
{
  company: {
    office: {
      location: {
        city: "Bangalore"
      }
    }
  }
}
```
But any level in the object can be missing.

YOUR TASK
- Access the city safely.
- Ensure no TypeError occurs.
- If any part is missing, return "Location Not Available".

CONTEXT: Any of the following parts may be missing: company → office → location → city
```
// Access city safely with a default value
const city = data?.company?.office?.location?.city
  ?? "Location Not Available";
```
SOLUTION GOAL: Return the city if available, otherwise return "Location Not Available" without throwing any TypeError.

01 SOLUTION
SOLUTION 1
```
const city = user?.profile?.address?.city
    ?? "Unknown City";
```
SOLUTION 2
```
const city = user?.profile?.address?.city;
if (city) {
  return city;
}
return "Unknown City";
```
SOLUTION 3
```
const city =
  user &&
  user.profile &&
  user.profile.address &&
  user.profile.address.city
  ? user.profile.address.city
  : "Unknown City";
```

02 SOLUTION
SOLUTION 1 | SOLUTION 2 | SOLUTION 3 | SOLUTION 4: [illegible]

03 SOLUTIONS
[illegible]

04 SOLUTIONS
[illegible]

05 SOLUTIONS
[illegible]

06 SOLUTION
[illegible]

07 SOLUTIONS
[illegible]

08 SOLUTIONS
[illegible]

09 SOLUTIONS
[illegible]